

文章笔记

# 文章笔记：为 AI Agent 编写高效工具

基于 Anthropic 工程博客整理

2026 年 4 月 2 日

## Writing Effective Tools for Agents — with Agents

Anthropic Engineering Blog

Agents are only as effective as the tools we give them.

We share how to write high-quality tools and evaluations,  
and how you can boost performance by using Claude to optimize its tools for itself.

文章作者：Ken Aizawa (Anthropic Engineering)

发布日期：2025-09-11

预计阅读时长：约 20 分钟

文章链接：<https://www.anthropic.com/engineering/writing-tools-for-agents>

项目主页：<https://github.com/hqh1025/ai-course-notes>

# 目录

|                                      |           |
|--------------------------------------|-----------|
| <b>1 引言：重新思考面向 Agent 的软件设计</b>       | <b>2</b>  |
| 1.1 Tool 的本质：确定性与非确定性的桥梁             | 2         |
| 1.2 本章小结                             | 2         |
| <b>2 工具开发的迭代流程</b>                   | <b>2</b>  |
| 2.1 第一步：快速搭建原型                       | 3         |
| 2.2 第二步：构建评估体系                       | 3         |
| 2.2.1 生成评估任务                         | 3         |
| 2.2.2 运行评估                           | 4         |
| 2.2.3 分析评估结果                         | 4         |
| 2.3 第三步：与 Agent 协同优化                 | 5         |
| 2.4 本章小结                             | 5         |
| <b>3 原则一：选择正确的工具集</b>                | <b>5</b>  |
| 3.1 Agent 的认知特性与传统软件的差异              | 5         |
| 3.2 工具整合的策略                          | 6         |
| 3.3 本章小结                             | 6         |
| <b>4 原则二：工具的命名空间</b>                 | <b>6</b>  |
| 4.1 Namespacing 策略                   | 6         |
| 4.2 Namespacing 的深层价值                | 7         |
| 4.3 本章小结                             | 7         |
| <b>5 原则三：返回有意义的上下文</b>               | <b>7</b>  |
| 5.1 避免低级技术标识符                        | 7         |
| 5.2 灵活的响应格式                          | 8         |
| 5.3 本章小结                             | 8         |
| <b>6 原则四：优化 Token 效率</b>             | <b>8</b>  |
| 6.1 控制响应大小的策略                        | 8         |
| 6.2 引导 Agent 的高效行为                   | 9         |
| 6.3 本章小结                             | 9         |
| <b>7 原则五：Prompt Engineering 工具描述</b> | <b>9</b>  |
| 7.1 编写工具描述的核心原则                      | 9         |
| 7.2 进阶资源                             | 10        |
| 7.3 本章小结                             | 10        |
| <b>8 五大原则的综合框架</b>                   | <b>10</b> |
| <b>9 总结与延伸</b>                       | <b>10</b> |
| 9.1 核心启示                             | 10        |
| 9.2 展望未来                             | 11        |
| 9.3 拓展阅读                             | 11        |

# 1 引言：重新思考面向 Agent 的软件设计

随着 Model Context Protocol (MCP) 等标准的出现，LLM Agent 可以通过数百个工具来完成现实世界的任务。但一个核心问题是：**如何让这些工具发挥最大效用？**

本文是 Anthropic 工程团队在实际开发中积累的工具设计最佳实践，涵盖了从原型搭建、评估体系到工具优化的全流程，最终提炼出五大核心原则。文章的一大亮点在于：**你可以让 Agent 自己来优化它所使用的工具。**

## 核心观点

Agent 的能力上限取决于我们赋予它的工具。为 Agent 设计工具与为人类开发者设计 API 有本质区别——我们需要从确定性系统（deterministic systems）转向为非确定性系统（non-deterministic agents）设计软件接口。

## 1.1 Tool 的本质：确定性与非确定性的桥梁

在传统软件工程中，我们编写的是**确定性系统之间的契约**。例如，调用 `getWeather("NYC")` 每次都会以完全相同的方式获取纽约的天气数据。

但 Tool 是一种**新型软件**，它反映的是确定性系统与非确定性 Agent 之间的契约。当用户问“今天需要带伞吗？”时，Agent 可能会调用天气工具，也可能从通用知识中回答，甚至可能先追问用户所在的城市。偶尔，Agent 还可能产生幻觉（hallucination）或根本不理解如何使用某个工具。

### 确定性 vs 非确定性系统

- **确定性系统**：相同输入总是产生相同输出。传统函数调用、API 请求都属于此类。
- **非确定性系统**：即使输入相同，也可能产生不同的输出。LLM Agent 就是典型代表。
- **Tool 的角色**：在两者之间搭建桥梁，让 Agent 能够与确定性的外部世界交互。

这意味着我们需要根本性地重新思考软件编写方式：不再像为其他开发者或系统编写函数和 API 那样编写工具和 MCP Server，而要**专门为 Agent 而设计**。

文章的核心目标是：扩大 Agent 能够有效解决问题的覆盖面，让 Agent 通过工具追求多种成功策略。一个有趣的发现是：对 Agent 最“符合人体工学”（ergonomic）的工具，往往对人类来说也出奇地直观。

## 1.2 本章小结

Tool 是连接确定性系统和非确定性 Agent 的新型软件接口。为 Agent 设计工具需要从传统 API 设计思维中跳脱出来，核心在于理解 Agent 的认知特性和行为模式，而非简单地暴露底层功能。

# 2 工具开发的迭代流程

Anthropic 提出了一套三步迭代流程：**原型搭建 → 评估测量 → 协同优化**。这一流程的核心理念是：工具质量无法靠一次性设计达到最优，必须通过系统化的评估和反复迭代来持续改进。

2.1 第一步：快速搭建原型

预测哪些工具对 Agent 来说是”好用的”、哪些不好用，往往需要实际动手测试。Anthropic 建议从快速原型开始：

- 1. **编写工具**：可以用 Claude Code 一次性 (one-shot) 生成工具代码。给 Claude 提供相关库、API、SDK 的文档会很有帮助，特别是 `llms.txt` 格式的文档。
- 2. **包装与连接**：将工具包装为本地 MCP Server 或 Desktop Extension (DXT)，这样可以直接在 Claude Code 或 Claude Desktop 中测试。
- 3. **手动测试**：亲自使用工具，识别粗糙之处 (rough edges)，收集用户反馈。

连接本地 MCP Server 到 Claude Code 的命令：

```
1 claude mcp add <name> <command> [args...]
```

Listing 1: 连接本地 MCP Server

工具也可以直接通过 Anthropic API 调用进行程序化测试，适用于自动化场景。

2.2 第二步：构建评估体系

原型搭好后，需要**系统化地衡量** Claude 使用工具的效果。这是整个流程中最关键的环节。

2.2.1 生成评估任务

评估任务应该**基于真实世界场景**，使用真实的数据源和服务（如内部知识库、微服务），避免过于简单的”沙盒”环境。强有力的评估任务通常需要多个工具调用——甚至可能数十个。

表 1: 强评估任务 vs 弱评估任务对比

| 强任务（推荐）                                                       | 弱任务（不推荐）                                      |
|---------------------------------------------------------------|-----------------------------------------------|
| 安排下周与 Jane 开会讨论 Acme 项目，附上上次项目规划会议的笔记，并预定会议室                  | 安排下周与 jane@acme.corp 开会                       |
| 客户 9182 反馈同一笔购买被扣款三次，查找所有相关日志，确定是否有其他客户受同一问题影响                | 在支付日志中搜索 purchase_complete 和 customer_id=9182 |
| 客户 Sarah Chen 提交了取消请求，准备挽留方案：(1) 离开原因 (2) 最有说服力的挽留方案 (3) 风险因素 | 查找客户 45892 的取消请求                              |

避免过度严格的验证器

验证器 (verifier) 不应因格式、标点或有效的替代表述等差异而拒绝正确答案。可以使用精确字符串匹配，也可以使用 LLM-as-Judge 方式让 Claude 来判断回答的正确性。但不要对 Agent 的求解策略做过多限定——同一任务可能存在多条正确路径。

2.2.2 运行评估

Anthropic 推荐用**程序化方式**（直接调用 LLM API）运行评估，每个评估任务使用一个简单的 agentic loop（while 循环交替执行 LLM API 调用和 Tool 调用）。

评估 Agent 的 System Prompt 设计

在评估 Agent 的 system prompt 中，建议指导 Agent 在工具调用和最终回答之前输出 **reasoning**（推理过程）和 **feedback**（反馈）块。这样做有两个好处：

- 触发 Chain-of-Thought (CoT) 行为，提升 LLM 的有效智能
- 帮助开发者理解 Agent 为什么调用或不调用某些工具

如果使用 Claude，可以直接开启 interleaved thinking 来获得类似效果。

除了顶层准确率，还应收集以下指标：

- 单个 Tool 调用和任务的总运行时间
- 总 Tool 调用次数
- 总 Token 消耗量
- Tool 错误率和错误类型

跟踪 Tool 调用可以揭示 Agent 常用的工作流，并发现工具整合优化的机会。

2.2.3 分析评估结果

如何从评估结果中提取洞察

Agent 是发现问题和提供反馈的好帮手，但要注意：**Agent 没有说出的内容，往往比它说出的更重要**。LLM 并不总是准确表达它的真实意图。分析评估结果时应：

- 观察 Agent 在哪些地方卡住或困惑
- 仔细阅读 reasoning/feedback 和 CoT 输出
- 审查原始 transcript（包括 Tool 调用和响应），捕捉 CoT 中未明确描述的行为
- ”读懂弦外之音”——评估 Agent 自己并不知道正确答案和最优策略

分析 Tool 调用指标可以发现具体问题：

表 2: 从指标异常定位工具问题

| 指标异常              | 可能的原因与改进方向         |
|-------------------|--------------------|
| 大量冗余 Tool 调用      | 分页或 Token 限制参数需要调整 |
| 大量无效参数导致的 Tool 错误 | 工具描述不够清晰，需要更好的示例   |
| 不必要的参数附加（如年份）     | 工具描述需要改进以引导正确行为    |

文章举了一个具体案例：在推出 Claude 的 Web Search 工具时，团队发现 Claude 会不必要地在查询参数后面追加“2025”，导致搜索结果偏差、性能下降。解决方法是改进工具描述(tool description)来引导 Claude 走向正确行为。

## 2.3 第三步：与 Agent 协同优化

这是本文最具创新性的部分：让 Agent 自己分析评估结果并改进工具。

具体做法是将评估 Agent 的 transcript 拼接在一起，粘贴到 Claude Code 中。Claude 擅长分析 transcript 并一次性重构大量工具——例如确保在引入新变更时，工具的实现与描述保持一致。

### Agent 自我优化的关键发现

Anthropic 团队发现，通过 held-out test set（留出测试集）验证，Agent 驱动优化能够带来超越“专家级”实现的性能提升——无论这些工具是由研究人员手写还是由 Claude 自身生成的。这表明评估驱动的迭代优化是系统性提升工具质量的有效方法。

## 2.4 本章小结

工具开发的最佳实践遵循“原型 → 评估 → 优化”的迭代循环。核心要点包括：(1) 评估任务必须贴近真实场景且有足够复杂度；(2) 收集多维度指标而非仅关注准确率；(3) Agent 本身就是强大的分析和优化工具，可以通过分析 transcript 来自动改进工具质量；(4) 使用 held-out test set 防止过拟合。

# 3 原则一：选择正确的工具集

更多的工具并不总是带来更好的结果。这一原则是文章所有原则中最具战略性的一条。

## 3.1 Agent 的认知特性与传统软件的差异

Agent 拥有与传统软件不同的 **affordance**（感知到的可操作性）。LLM Agent 的“上下文”是有限且昂贵的，而传统计算机内存则廉价且充足。

文章用**通讯录搜索**来类比说明这一差异：

表 3: 通讯录搜索：传统软件 vs Agent

|      | 传统软件                           | LLM Agent                         |
|------|--------------------------------|-----------------------------------|
| 方法   | 遍历联系人列表，逐个检查                   | 必须将信息加载到有限的上下文窗口中                 |
| 成本   | 内存廉价，遍历高效                      | 每个 Token 都消耗上下文空间                 |
| 最佳策略 | <code>list_contacts</code> 后过滤 | <code>search_contacts</code> 直接定位 |

常见反模式：简单包装已有 API

最常见的错误是将已有的软件功能或 API 端点简单包装为 Tool，而不考虑这些工具是否适合 Agent 使用。这相当于强迫 Agent 用”逐页阅读通讯录”的方式来查找联系人——即暴力搜索 (brute-force search)，而自然的方式是直接按字母跳转到相关页面。

3.2 工具整合的策略

工具可以整合功能，在底层处理多个离散操作或 API 调用。文章给出了三组对比：

表 4: 工具设计：碎片化 vs 整合

| 碎片化设计（不推荐）                                          | 整合设计（推荐）                             |
|-----------------------------------------------------|--------------------------------------|
| list_users + list_events + create_event             | schedule_event: 自动查找可用时间并安排事件        |
| read_logs                                           | search_logs: 只返回相关日志行及上下文            |
| get_customer_by_id + list_transactions + list_notes | get_customer_context: 一次性编译客户的所有相关信息 |

每个工具应有**清晰、独特的用途**。工具应该让 Agent 以类似人类的方式分解和解决任务，同时减少中间输出对上下文的消耗。

过多工具或功能重叠的工具会分散 Agent 的注意力，使其偏离高效策略。精心、有选择地规划工具集可以带来显著的回报。

3.3 本章小结

选择正确的工具集是一个战略性决策。核心原则是：(1) 不要简单包装已有 API；(2) 优先实现面向高影响力工作流的少量精心设计的工具；(3) 通过工具整合减少 Agent 的认知负担和上下文消耗；(4) 确保每个工具有清晰独特的用途。

4 原则二：工具的命名空间

当 Agent 同时访问数十个 MCP Server 和数百个工具时，功能重叠或用途模糊的工具会导致 Agent 困惑，不知道该使用哪一个。

4.1 Namespacing 策略

**命名空间** (Namespacing) 通过将相关工具归类到共同前缀下来划定边界，MCP 客户端有时会默认执行此操作。

表 5: 命名空间的两个维度

| 维度    | 示例                                           | 效果          |
|-------|----------------------------------------------|-------------|
| 按服务分组 | asana_search, jira_search                    | 区分不同平台的同类操作 |
| 按资源分组 | asana_projects_search,<br>asana_users_search | 区分同一平台的不同资源 |

### 前缀式 vs 后缀式命名

Anthropic 发现，选择前缀式 (asana\_search) 还是后缀式 (search\_asana) 命名对评估结果有**非平凡的影响** (non-trivial effects)。效果因 LLM 而异，建议根据自己的评估结果来选择命名方案。

## 4.2 Namespacing 的深层价值

命名空间带来的好处不仅仅是组织性，更在于它可以：

1. **减少加载到 Agent 上下文中的工具数量和描述**：通过名称反映任务的自然分工
2. **将 Agent 的认知计算卸载到 Tool 调用本身**：减少 Agent 在上下文中的推理负担
3. **降低 Agent 犯错的整体风险**：错误包括调用错误的工具、使用错误的参数、调用工具太少、或错误处理工具响应

## 4.3 本章小结

合理的命名空间策略能帮助 Agent 在大量工具中快速定位正确选择。按服务和资源两个维度组织工具名称，并通过评估来选择最优的命名方案。

# 5 原则三：返回有意义的上下文

工具的返回值设计同样至关重要。工具实现应该只返回**高信号** (high signal) 的信息给 Agent，优先考虑上下文相关性而非灵活性。

## 5.1 避免低级技术标识符

表 6: 返回字段设计：低信号 vs 高信号

| 低信号字段（避免）       | 高信号字段（推荐） |
|-----------------|-----------|
| uuid            | name      |
| 256px_image_url | image_url |
| mime_type       | file_type |

Agent 处理自然语言的名称、术语或标识符的能力**远优于**处理密码学标识符。Anthropic 发现，仅将任意字母数字 UUID 解析为更具语义意义的可解释语言（甚至是从 0 开始的 ID 方案），就能显著提高 Claude 在检索任务中的**精确度**，减少幻觉。

## 5.2 灵活的响应格式

在某些场景下，Agent 可能需要同时使用自然语言标识符和技术标识符来触发下游 Tool 调用。例如：

```
search_user(name='jane') → send_message(id=12345)
```

解决方案是暴露一个简单的 `response_format` 枚举参数，让 Agent 自行控制工具返回”简洁” (concise) 还是”详细” (detailed) 响应：

```
1 from enum import Enum
2
3 class ResponseFormat(Enum):
4     DETAILED = "detailed" # 返回完整信息
5     CONCISE = "concise"   # 仅返回必要标识符
```

Listing 2: ResponseFormat 枚举示例

还可以添加更多格式以获得更大灵活性，类似于 GraphQL 中选择性请求特定字段的能力。

### 响应结构的选择没有银弹

工具响应的结构——XML、JSON 还是 Markdown——对评估性能有实际影响。LLM 基于 next-token prediction 训练，倾向于更好地处理与训练数据匹配的格式。没有一种万能的解决方案，最优的响应结构因任务和 Agent 而异，需要通过评估来确定。

## 5.3 本章小结

工具返回值应追求高信号、语义化。避免返回低级技术标识符，优先使用自然语言和可解释的命名。通过 `response_format` 参数提供灵活性，并根据评估选择最优的响应结构格式。

## 6 原则四：优化 Token 效率

优化上下文的质量很重要，但优化返回给 Agent 的上下文数量同样关键。

### 6.1 控制响应大小的策略

对于可能消耗大量上下文的 Tool 响应，建议实现以下机制的组合：

表 7: 控制 Tool 响应大小的四种策略

| 策略                     | 说明                      |
|------------------------|-------------------------|
| 分页 (Pagination)        | 将大量结果分批返回，Agent 按需请求下一页 |
| 范围选择 (Range Selection) | 允许 Agent 指定返回结果的范围      |
| 过滤 (Filtering)         | 允许 Agent 通过条件过滤只获取相关结果  |
| 截断 (Truncation)        | 对超长响应自动截断，并附带有用的引导信息    |

### Claude Code 的 Token 限制实践

在 Claude Code 中, Anthropic 默认将 Tool 响应限制为 **25,000 tokens**。虽然他们预计 Agent 的有效上下文长度会随时间增长, 但对 Token 高效工具的需求将始终存在。

## 6.2 引导 Agent 的高效行为

如果选择截断响应, 请确保附带有用的引导指令。可以:

- 直接鼓励 Agent 采用更 Token 高效的策略, 例如进行多次小型定向搜索, 而非一次大范围搜索
- 在错误响应中清晰传达具体、可操作的改进建议, 而非返回不透明的错误代码或调用栈

### 错误响应也是 Prompt Engineering

Tool 的截断和错误响应本身就是一种 prompt engineering。它们可以引导 Agent 采用更 Token 高效的工具使用行为 (如使用过滤器或分页), 或展示正确格式化的工具输入示例。将错误响应视为与 Agent 沟通的机会, 而非简单的失败通知。

## 6.3 本章小结

Token 效率直接影响 Agent 的表现。通过分页、过滤、截断等策略控制响应大小, 并利用截断和错误响应来引导 Agent 走向高效行为。Claude Code 的 25,000 token 默认限制是一个实用参考。

## 7 原则五: Prompt Engineering 工具描述

这是最后一个、也是**最有效**的方法: 对工具的描述和规格 (descriptions and specs) 进行 Prompt Engineering。因为这些描述会被加载到 Agent 的上下文中, 它们可以系统性地引导 Agent 走向有效的工具调用行为。

### 7.1 编写工具描述的核心原则

编写工具描述时, 应该像**向团队新人介绍你的工具**一样思考。考虑你可能隐含带入的上下文——专用查询格式、小众术语的定义、底层资源之间的关系——并将这些**显式化**。

具体建议:

1. **消除歧义**: 用严格的数据模型明确描述 (并强制执行) 预期的输入和输出
2. **参数命名清晰**: 不要用 `user` 这样的模糊名称, 使用 `user_id` 明确指代
3. **提供充足的上下文**: 专用格式、术语定义、资源关系都应在描述中说明
4. **结合评估验证**: 即使是微小的描述优化也能带来显著的性能提升

### 实战案例: SWE-bench 上的突破

Claude Sonnet 3.5 在 SWE-bench Verified 评估上取得了当时的 state-of-the-art 性能, 而关键手段之一就是**对工具描述进行精确的优化**。这些优化大幅降低了错误率并提高了任务完成率, 证明了 prompt engineering 工具描述的巨大潜力。

## 7.2 进阶资源

Anthropic 还推荐了以下进阶内容：

- **Developer Guide**：更多工具定义的最佳实践
- **工具动态加载机制**：了解工具如何被动态加载到 Claude 的 system prompt 中
- **Tool Annotations (MCP Server)**：声明哪些工具需要开放世界访问或执行破坏性操作

## 7.3 本章小结

工具描述的 Prompt Engineering 是最具杠杆效应的优化手段。像对待新人入职文档一样编写工具描述，将隐式知识显式化，消除歧义，并通过评估验证每次优化的效果。即使微小的改进也可能带来戏剧性的性能提升。

## 8 五大原则的综合框架

将文章的五大原则整合来看，它们共同构成了一个完整的工具设计框架：

表 8: 五大原则一览

| 编号 | 原则                    | 核心要点                     |
|----|-----------------------|--------------------------|
| 1  | 选择正确的工具集              | 少而精，面向工作流，整合多步操作         |
| 2  | 工具命名空间                | 按服务和资源分层命名，帮助 Agent 快速定位 |
| 3  | 返回有意义的上下文             | 高信号、语义化，避免低级标识符          |
| 4  | 优化 Token 效率           | 分页、过滤、截断，引导高效行为          |
| 5  | Prompt Engineering 描述 | 显式化隐含知识，消除歧义，结合评估迭代      |

### 各原则之间的相互作用

五大原则并非独立运作，而是相互强化的。例如：选择了正确的工具集（原则 1）会减少命名空间的复杂度（原则 2）；优化返回值（原则 3）本身就是 Token 效率优化（原则 4）的一部分；而所有这些决策都需要通过 Prompt Engineering（原则 5）来正确传达给 Agent。如果只优化其中一个维度而忽略其他，整体效果可能会打折扣。

这些原则的共同主题是：**以 Agent 的认知模型为中心进行设计**，而不是以底层系统的技术架构为中心。Agent 的上下文窗口是稀缺资源，每一个设计决策都应该围绕“如何让 Agent 用最少的上下文做最多的事”来展开。

## 9 总结与延伸

### 9.1 核心启示

Anthropic 在这篇工程博客中传递的最重要信息是：为 Agent 构建高效工具需要从传统确定性软件开发范式转向面向非确定性系统的新范式。

通过评估驱动的迭代过程，Anthropic 团队识别出了成功工具的一致模式：

1. 有效的工具被**有意识地、清晰地定义**
2. 它们**审慎地使用 Agent 上下文**
3. 它们可以在**多样化的工作流中灵活组合**
4. 它们使 Agent 能够**直觉地解决现实世界任务**

## 9.2 展望未来

文章预期 Agent 与世界交互的具体机制将不断演化——从 MCP 协议的更新到底层 LLM 自身的升级。但评估驱动的工具优化方法论将保持其价值：随着 Agent 变得更加强大，与之配合的工具也将随之进化。

## 9.3 拓展阅读

- Anthropic Developer Guide: Tool Use Best Practices  
<https://docs.anthropic.com/en/docs/build-with-claude/tool-use>
- Model Context Protocol (MCP) 官方文档  
<https://modelcontextprotocol.io/>
- Anthropic Tool Evaluation Cookbook  
<https://github.com/anthropics/anthropic-cookbook>
- SWE-bench Verified 评测  
<https://www.swebench.com/>
- Claude Code 文档  
<https://docs.anthropic.com/en/docs/claude-code>